

机器学习实验指导手册

交叉验证与模型选择

一、实验目的

- 掌握交叉验证的基本原理：**理解 k 折交叉验证、分层交叉验证等方法的设计思想与应用场景。
- 学会使用 `scikit-learn` 实现交叉验证：**熟练调用工具包中的 `cross_val_score`、`GridSearchCV` 等接口。
- 掌握模型选择的核心方法：**通过交叉验证对比不同模型的性能，优化超参数选择。
- 培养解决实际问题的能力：**针对真实数据集，设计完整的模型选择与验证流程。

二、实验原理

1. 基本概念

(1) 定义

交叉验证是一种统计学方法，用于评估机器学习模型的泛化性能。其核心思想是**重复使用数据**，将数据集划分为训练集和验证集多次，从而减少因单次数据划分带来的偏差。

(2) 解决的问题

- 避免模型因单次数据划分（如 `train_test_split`）的随机性导致评估结果不稳定。
- 通过多次划分训练集与验证集，减少模型评估结果的方差。
- 充分利用有限数据，尤其在小规模数据集上更有效。

2. 交叉验证常用方法及原理

(1) k 折交叉验证 (k-Fold CV)

原理：

- 将数据集均等分为 k 个子集（称为“折”）。

- b) 每次使用其中 **k-1 折** 作为训练集，**剩余 1 折** 作为验证集。
- c) 重复 k 次，确保每折数据均被用作验证集一次。
- d) 最终结果为 k 次评估的**平均值**。

特点：

- 最常用的方法（通常 k=5 或 10）。
- 数据利用率高，适合中小规模数据集。

(2) 留一交叉验证 (Leave-One-Out, LOO)

原理：

- k=样本数 N，即每次仅留一个**样本**作为验证集。
- 计算开销大，但适合极小型数据集。

(3) 分层 k 折交叉验证 (Stratified k-Fold)

原理：

- 在 k 折基础上，**保持每折中类别比例**与原始数据集一致。
- 适用于分类任务中类别不平衡的场景。

3. 模型选择与超参数优化

- **网格搜索 (Grid Search)**：遍历所有候选参数组合，通过交叉验证选择最优参数。
- **随机搜索 (Random Search)**：在参数空间中随机采样，适用于高维参数空间。

三、实验步骤

1. 数据准备

使用 `scikit-learn` 内置的乳腺癌数据集：

```
from sklearn.datasets import load_breast_cancer  
  
data = load_breast_cancer()  
  
X, y = data.data, data.target
```

2. 数据预处理

标准化数据并划分训练集与测试集：

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)
```

3. 交叉验证实现

k 折交叉验证示例：

```
from sklearn.model_selection import cross_val_score
from sklearn.svm import SVC
model = SVC(kernel='linear')
scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')
print(f"交叉验证平均准确率： {scores.mean():.2f} (±{scores.std():.2f})")
```

分层交叉验证示例：

```
from sklearn.model_selection import StratifiedKFold
stratified_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X_train, y_train, cv=stratified_cv)
print(f"分层交叉验证平均准确率： {scores.mean():.2f}")
```

4. 模型选择与超参数优化

使用网格搜索选择 SVM 最优参数：

```
from sklearn.model_selection import GridSearchCV
param_grid = { 'C': [0.1, 1, 10], 'gamma': [0.01, 0.1, 1], 'kernel': ['rbf', 'linear'] }
grid_search = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)
print("最优参数组合： ", grid_search.best_params_)
print("最优模型验证准确率： ", grid_search.best_score_)
```

5. 模型评估

在独立测试集上验证最优模型性能：

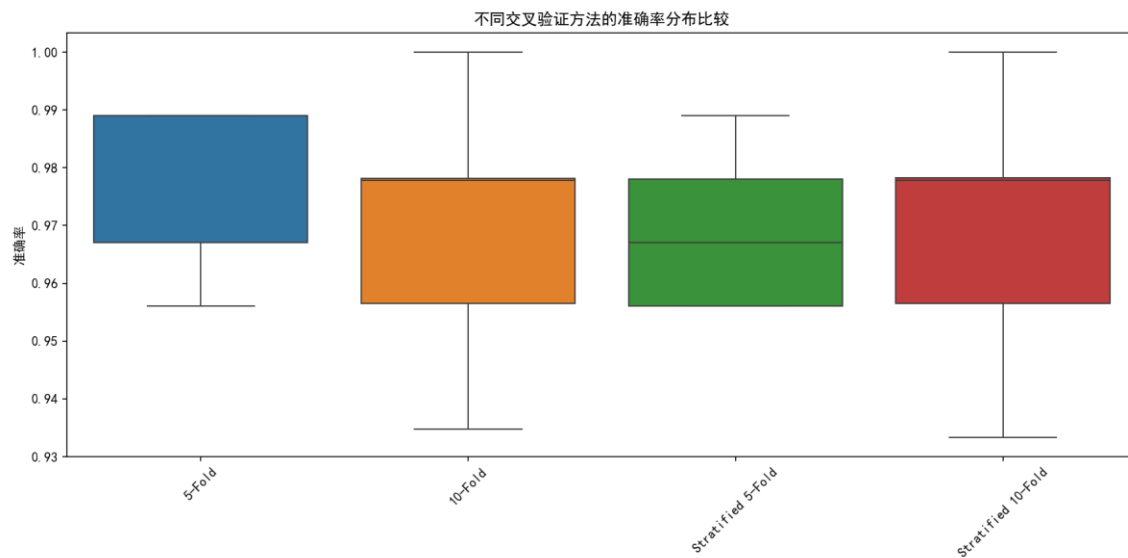
```
best_model = grid_search.best_estimator_  
y_pred = best_model.predict(X_test)  
print(f"测试集准确率: {accuracy_score(y_test, y_pred):.2f}")
```

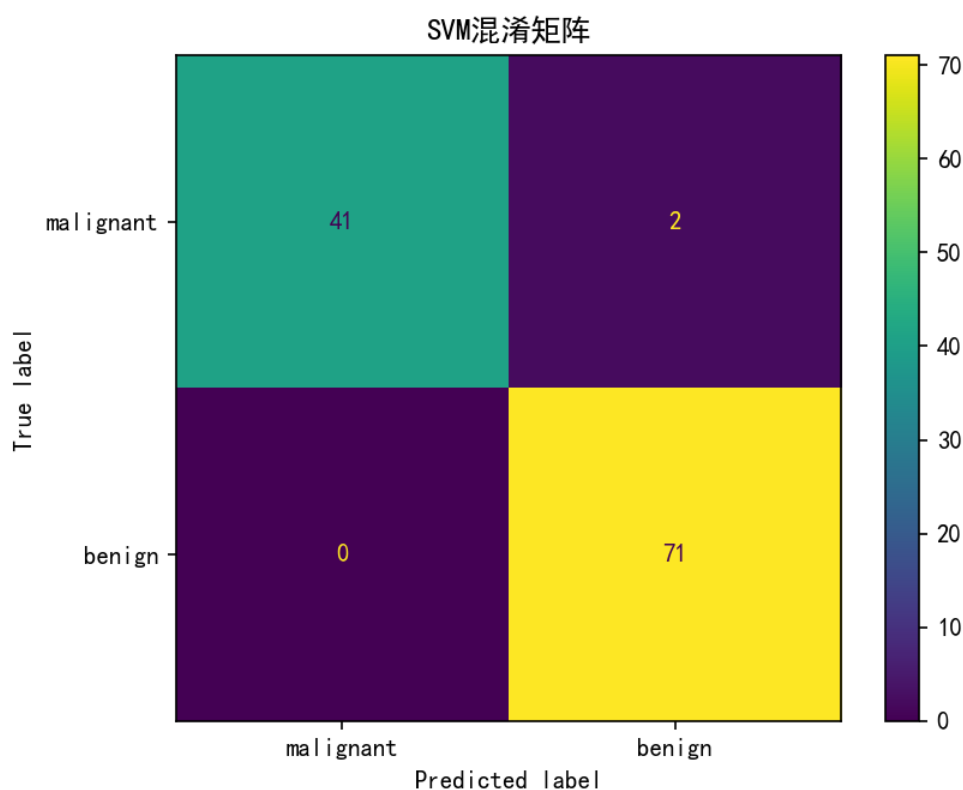
四、实验要求

必做任务

- 交叉验证对比实验：分别实现 5 折、10 折交叉验证，比较两者在相同模型下的性能差异。

```
数据集形状: (569, 30)  
特征数量: 30  
类别分布:  
1    357  
0    212  
Name: count, dtype: int64  
类别名称: ['malignant' 'benign']  
5-Fold交叉验证结果: 平均准确率=0.9780 ( $\pm 0.0139$ )  
10-Fold交叉验证结果: 平均准确率=0.9715 ( $\pm 0.0196$ )  
Stratified 5-Fold交叉验证结果: 平均准确率=0.9692 ( $\pm 0.0128$ )  
Stratified 10-Fold交叉验证结果: 平均准确率=0.9715 ( $\pm 0.0198$ )
```





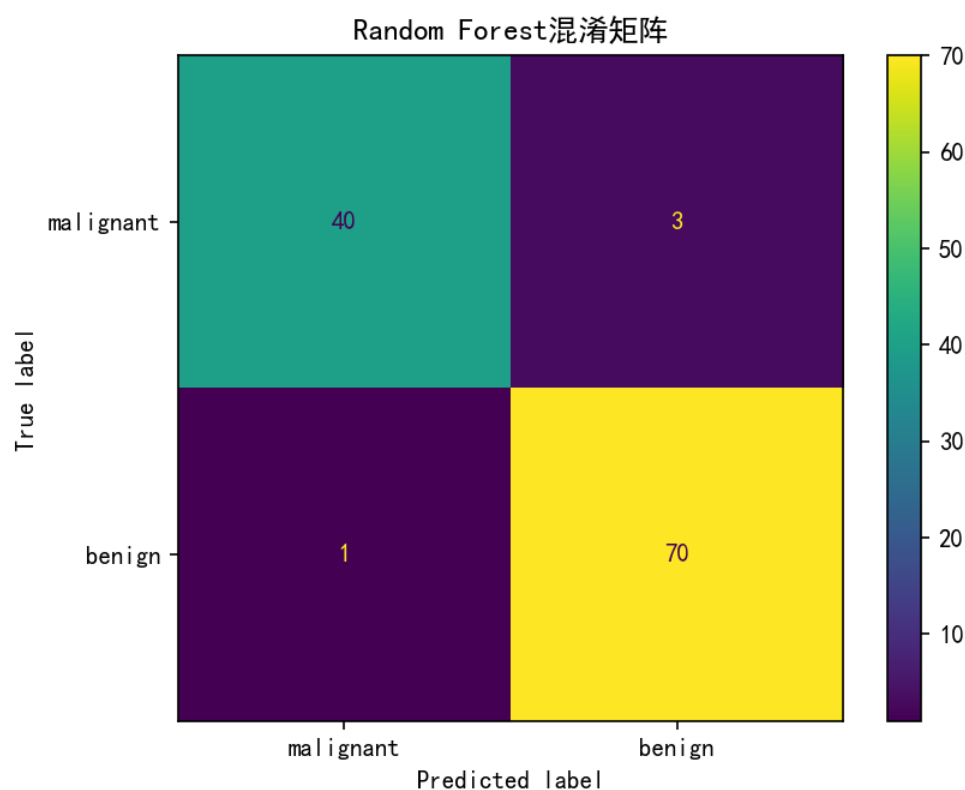
SVM网格搜索结果：
最优参数组合：{'C': 10, 'gamma': 0.01, 'kernel': 'rbf'}
最优模型交叉验证准确率：0.9780

SVM随机搜索结果：
最优参数组合：{'kernel': 'rbf', 'gamma': 0.001, 'C': 100}
最优模型交叉验证准确率：0.9736

随机森林网格搜索结果：
最优参数组合：{'max_depth': 5, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
最优模型交叉验证准确率：0.9560

SVM模型评估：
交叉验证准确率：0.9780 (±0.0155)
测试集准确率：0.9825
分类报告：

	precision	recall	f1-score	support
malignant	1.00	0.95	0.98	43
benign	0.97	1.00	0.99	71
accuracy			0.98	114
macro avg	0.99	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114



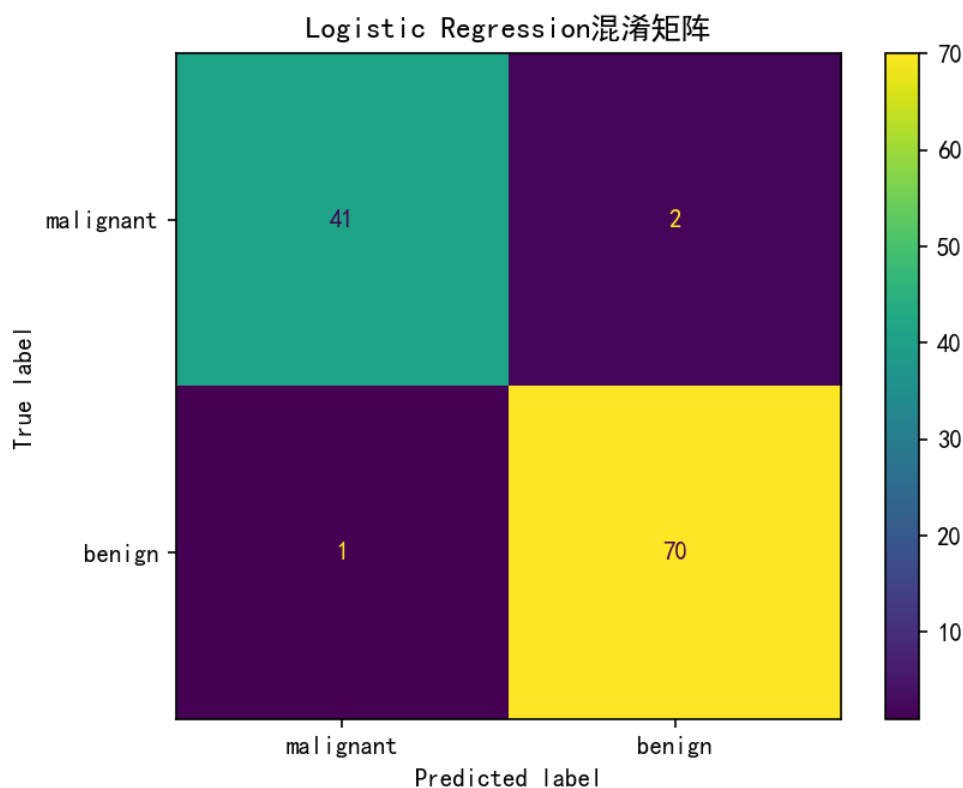
Random Forest模型评估:

交叉验证准确率: 0.9560 (± 0.0220)

测试集准确率: 0.9649

分类报告:

	precision	recall	f1-score	support
malignant	0.98	0.93	0.95	43
benign	0.96	0.99	0.97	71
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114



Logistic Regression模型评估:

交叉验证准确率: 0.9626 (± 0.0179)

测试集准确率: 0.9737

分类报告:

	precision	recall	f1-score	support
malignant	0.98	0.95	0.96	43
benign	0.97	0.99	0.98	71
accuracy			0.97	114
macro avg	0.97	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

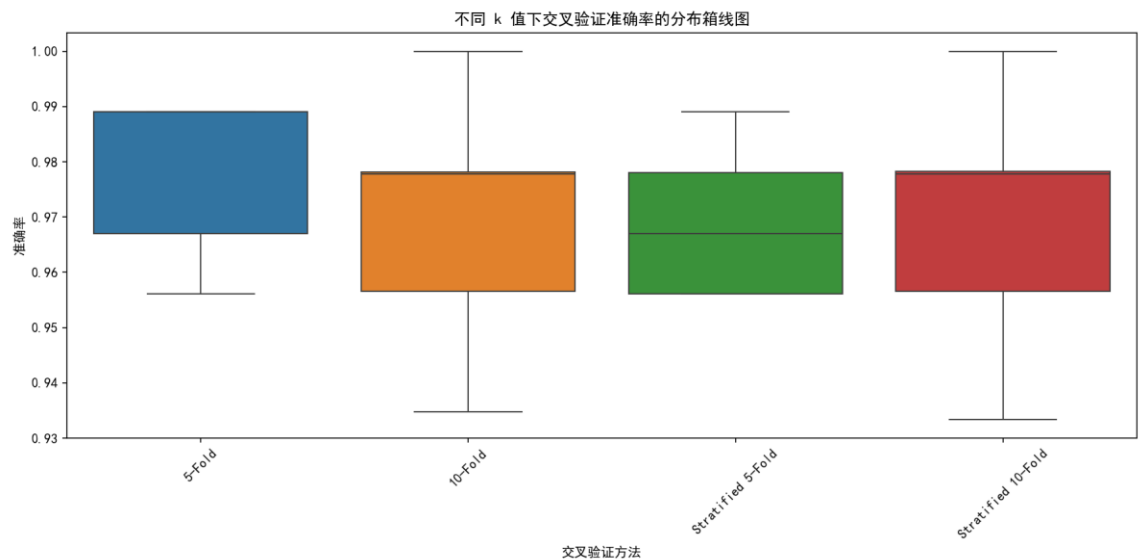
- 网格搜索调参: 对随机森林模型的 `n_estimators` 和 `max_depth` 参数进行网格搜索, 输出最优参数。

随机森林网格搜索结果:

最优参数组合: `{ 'max_depth': 5, 'n_estimators': 100 }`

最优模型交叉验证准确率: 0.9560

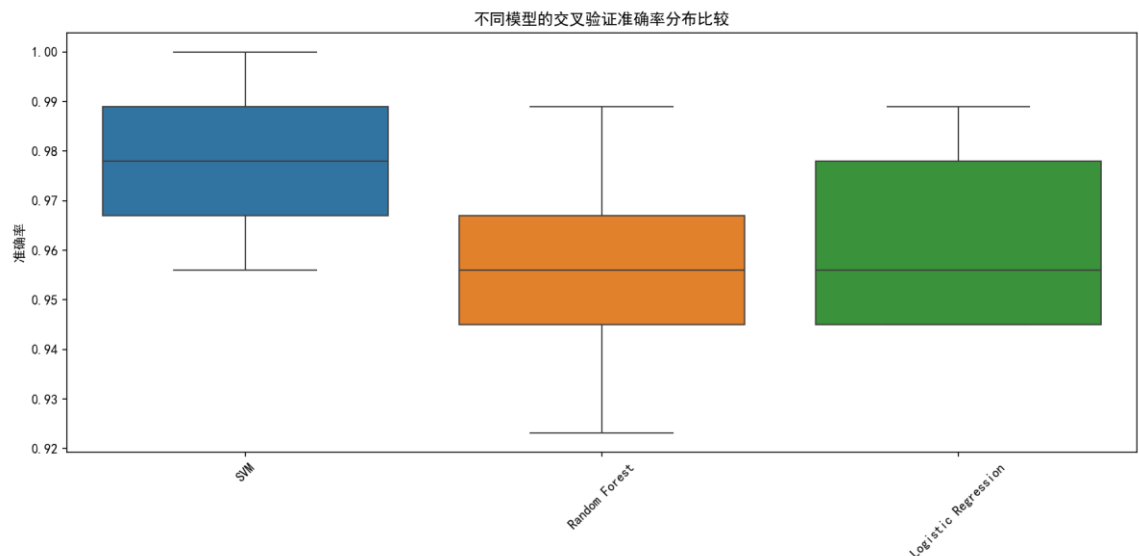
- 可视化分析：绘制不同 k 值下交叉验证准确率的分布箱线图。



●

扩展任务

- 随机搜索对比：使用 `RandomizedSearchCV` 优化 SVM 参数，对比其与网格搜索的效率与效果。
- 自定义交叉验证策略：实现按时间序列分割的交叉验证（例如前 4 折为历史数据，最后一折为近期数据）。



时间序列交叉验证结果：
平均准确率：0.9736 (± 0.0164)

- 多模型对比：比较 SVM、随机森林、逻辑回归在交叉验证中的表现，撰写分析报告。

五、实验分析

测试案例

数据集：乳腺癌数据集（二分类）

模型：SVM、随机森林

参数网格：

SVM: {'C': [0.1, 1, 10], 'kernel': ['linear', 'rbf']}

随机森林: {'n_estimators': [50, 100], 'max_depth': [5, 10]}

预期结果

网格搜索输出最优参数组合及对应准确率。

交叉验证结果需展示均值和标准差，例如：SVM 5 折交叉验证准确率：0.96 (± 0.02)

六、思考题

- 交叉验证的局限性：当数据集样本量极小时，k 折交叉验证是否仍然有效？请提出改进方案。

无效，改进方案：1.使用分层 k 折 2.用更大数据量同时使用 shuffle

- 过拟合风险：网格搜索中选择的“最优参数”是否可能在独立测试集上表现不佳？如何避免？

是，用 `fit(X_train, y_train)` 调整，并输出评估结果

- 计算效率：分析网格搜索与随机搜索的时间复杂度差异，如何权衡两者？
网格搜索需遍历所有参数组合（如 100 种），随机搜索仅尝试固定数量（如 10 种）。

随机搜索：快速粗筛，适合参数空间大或资源有限。**网格搜索**：精细优化，适合关键参数的小规模搜索。

- 实际应用：在工业级机器学习系统中，为什么常将交叉验证与独立测试集结合使用？

交叉验证：在训练集内部分块验证，确保参数调优的稳定性。**独立测试集**：用未参与训练的数据最终验证，防止模型对验证数据过拟合。**结合原因**：两者共同确保模型既调得好（交叉验证）又用得好（测试集）。

提示：调试时可利用 `sklearn.metrics` 中的 `classification_report` 输出详细评估结果，例如：

```
from sklearn.metrics import classification_report  
print(classification_report(y_test, y_pred))
```